

Hablando en ceros y unos

Binario, codificación y representación

Semana 2 · Introducción al Análisis de Datos y Programación

Horacio Samaniego (horaciosamaniego@uach.cl)

*Laboratorio de Ecoinformática · Instituto de Conservación, Biodiversidad y Territorio
Valdivia, UACH*

2026

Recuerdo de la Semana 1

La semana pasada:

- La memoria almacenaba "valores" en celdas
- La CPU operaba con "números"

Hoy respondemos: Semana 2: Intro al análisis de datos y programación · El sistema binario (CBIT010)

Hoja de ruta

1. **Truco** — Las cartas binarias
2. **Sistemas numéricos** — Decimal, binario, hexadecimal
3. **Texto** — ASCII y UTF-8
4. **Imágenes** — Píxeles y RGB
5. **Sonido** — Muestreo
6.  **Laboratorio** — Pulseras binarias
7.  **Control 1**



El truco de las cartas binarias

Las 5 cartas

Carta 1: 16 puntos ●●●●●●●●●●●●●●●●
Carta 2: 8 puntos ●●●●●●●●
Carta 3: 4 puntos ●●●●
Carta 4: 2 puntos ●●
Carta 5: 1 punto ●

Un voluntario piensa un número del **0 al 31**.
El curso voltea las cartas para representarlo.
Yo adivino.

El secreto: potencias de 2

Cada carta = una **potencia de 2**.

Cara arriba = **1**, cara abajo = **0**.

Ejemplo: el número **19**

- 16  + 8  + 4  + 2  + 1 
- Cartas: arriba, abajo, abajo, arriba, arriba
- En binario: **10011**

“Esto es **exactamente** lo que hace la memoria de un computador. Cada celda: encendida o apagada.”

Sistemas numéricos

Decimal, binario, hexadecimal

El sistema decimal (base 10)

Lo que ya saben — 10 símbolos: **0 1 2 3 4 5 6 7 8 9**

Cada posición = una **potencia de 10**:

$$347 = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 = 300 + 40 + 7$$

“No tiene nada de especial. Lo usamos porque tenemos 10 dedos.”

El sistema binario (base 2)

Dos símbolos: **0** y **1**

Cada posición = una **potencia de 2**:

Posición	8	7	6	5	4	3	2	1	0
Valor	256	128	64	32	16	8	4	2	1

- 1 bit = un dígito binario (0 o 1)
- **8 bits = 1 byte**
- 1 byte puede representar 256 valores (0–255)

Conversión: binario → decimal

Sumar las posiciones donde hay un 1:

$$10110 = 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1$$

$$= 16 + 4 + 2 = 22$$

Ejercicio rápido:

$$11001 = ?$$

--

$$11001 = 16 + 8 + 1 = 25 \quad \checkmark$$

Conversión: decimal → binario

Método de la resta — ir de la potencia más grande a la más pequeña:

Convertir **45** a binario:

- ¿128? No → 0. ¿64? No → 0. ¿32? Sí → resta 13 → **1**
- ¿16? No → 0. ¿8? Sí → resta 5 → **1**
- ¿4? Sí → resta 1 → **1**. ¿2? No → 0. ¿1? Sí → **1**

Resultado: **00101101**

Ejercicio: convertir **13** a binario

--

13 = 8 + 4 + 1 = 00001101 ✓

¿Por qué binario?

Un transistor tiene dos estados: conduce o no conduce. Un cable tiene voltaje alto o bajo. Con solo dos estados se minimiza el error por ruido eléctrico.

Es como clasificar hábitat en presente/ausente en vez de una escala 1–10: más tosco, pero mucho más robusto.

Hexadecimal (base 16) — versión compacta

16 símbolos: **0–9** y **A–F** (A=10, B=11, ..., F=15)

Cada dígito hex = exactamente **4 bits**:

Hex	Bin	Dec		Hex	Bin	Dec
0	0000	0		8	1000	8
5	0101	5		A	1010	10
7	0111	7		F	1111	15

Ejemplo: **FF** = **11111111** = **255**

Se usa en: colores web, direcciones de memoria, redes.

Hexadecimal en la vida real

El color blanco en una página web:

#FFFFFF = tres bytes: FF FF FF

= Rojo **255**, Verde **255**, Azul **255**

(Más sobre esto en unos minutos.)

Representando texto

ASCII y UTF-8

El problema

Un computador **solo almacena números** (secuencias de bits).

Para almacenar texto, necesitamos un **acuerdo**:

¿qué número corresponde a qué letra?

Ese acuerdo se llama **codificación de caracteres**.

ASCII (1963)

American Standard Code for Information Interchange

- 7 bits → **128 caracteres** posibles (0–127)
- Letras, dígitos, signos, caracteres de control

Carácter	Decimal	Binario
A	65	01000001
B	66	01000010
Z	90	01011010
a	97	01100001
0 (dígito)	48	00110000
espacio	32	00100000

ASCII (1963)

0 NUL	16 DLE	32	48 0	64 @	80 P	96 `	112 p
1 SOH	17 DC1	33 !	49 1	65 A	81 Q	97 a	113 q
2 STX	18 DC2	34 "	50 2	66 B	82 R	98 b	114 r
3 ETX	19 DC3	35 #	51 3	67 C	83 S	99 c	115 s
4 EOT	20 DC4	36 \$	52 4	68 D	84 T	100 d	116 t
5 ENQ	21 NAK	37 %	53 5	69 E	85 U	101 e	117 u
6 ACK	22 SYN	38 &	54 6	70 F	86 V	102 f	118 v
7 BEL	23 ETB	39 '	55 7	71 G	87 W	103 g	119 w
8 BS	24 CAN	40 (	56 8	72 H	88 X	104 h	120 x
9 HT	25 EM	41)	57 9	73 I	89 Y	105 i	121 y
10 LF	26 SUB	42 *	58 :	74 J	90 Z	106 j	122 z
11 VT	27 ESC	43 +	59 ;	75 K	91 [	107 k	123 {
12 FF	28 FS	44 ,	60 <	76 L	92 \	108 l	124
13 CR	29 GS	45 -	61 =	77 M	93]	109 m	125 }
14 SO	30 RS	46 .	62 >	78 N	94 ^	110 n	126 ~
15 SI	31 US	47 /	63 ?	79 O	95 _	111 o	127 DEL

Decimal - Binary - Octal - Hex – ASCII Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	\$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

This work is licensed under the Creative Commons Attribution-ShareAlike License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>

ASCII Conversion Chart.doc Copyright © 2008, 2012 Donald Weiman 22 March 2012

Un detalle elegante

Entre '**A**' (65) y '**a**' (97) hay exactamente **32** de diferencia.

$32 = 2^5 \rightarrow$ basta cambiar **un solo bit** (el bit 5) para pasar de mayúscula a minúscula.

```
A = 01000001
a = 01100001
    ↑ este bit
```

No es casualidad — es diseño inteligente.

🤔 ¿Qué pasa con la ñ? ¿Y con los acentos?

No están en ASCII. ASCII fue diseñado para el inglés.

UTF-8 — el estándar universal

- **Unicode:** catálogo de ~150.000 caracteres (todos los idiomas, emojis, símbolos)
- **UTF-8:** la codificación más usada en internet
- Usa **1 a 4 bytes** por carácter
- Los primeros 128 = idénticos a ASCII (compatible)

Ejemplos: ñ = 241, á = 225, 🍓 = U+1F333 (4 bytes)

“ Cuando ven "**Ã±**" en vez de "**ñ**" en una web, es un error de codificación — el navegador usó la tabla equivocada. ”

Representando imágenes

Píxeles y RGB

¿Qué es una imagen digital?

Una **grilla de puntos** (píxeles). Cada píxel tiene un color.

Resolución = ancho × alto

Ejemplo: $1920 \times 1080 = \sim 2$ millones de píxeles

(Demostración: hacer zoom a una foto hasta ver los píxeles.)

“ Toda foto, por hermosa que sea, es en el fondo una **tabla de números**. ”

Color RGB

Cada píxel = **3 valores**: Rojo, Verde, Azul

Cada canal: **1 byte** (0–255). Total: **3 bytes = 24 bits** por píxel.

Color	R	G	B	Hex
Negro	0	0	0	#000000
Blanco	255	255	255	#FFFFFF
Rojo puro	255	0	0	#FF0000
Verde bosque	27	94	32	#1B5E20
Azul océano	21	101	192	#1565C0

Colores posibles: $256^3 = 16.777.216$

¿Cuánto pesa una foto?

Foto de **12 megapíxeles** sin comprimir:

12.000.000 píxeles × 3 bytes = **36 MB**

Por eso existen JPEG y PNG: **compresión**.

“**Conexión ecológica:** las imágenes satelitales (Landsat, Sentinel) son lo mismo — grillas de píxeles. Pero con más canales: infrarrojo cercano, infrarrojo de onda corta... Clasificar uso de suelo es clasificar patrones de números en una tabla gigante.”

Representando sonido

Muestreo y cuantización

Del aire a los bits

El sonido es una **onda continua** (presión del aire).

Para digitalizarlo:

1. **Muestreo:** medir la amplitud a intervalos regulares
2. **Cuantización:** convertir cada medición en un número

Parámetros clave:

- **Frecuencia de muestreo:** CD = 44.100 Hz (44.100 mediciones/segundo)
- **Profundidad de bits:** CD = 16 bits (65.536 niveles posibles)

“**Ecología:** el monitoreo acústico de biodiversidad (AudioMoth) hace esto. Un AudioMoth a 48 kHz / 16 bits genera ~5 MB por minuto.”

Todo es bits

El resumen

Todo es convención

Dato	Codificación	Estándar
Números	Posicional binario	Complemento a 2, IEEE 754
Texto	Carácter → número → bits	ASCII, UTF-8
Imágenes	Píxel → 3 números (RGB) → bits	JPEG, PNG
Sonido	Muestras → números → bits	WAV, MP3

El hardware solo distingue **dos estados**.

Todo lo demás es un acuerdo humano sobre cómo interpretar las secuencias de bits. Sin el acuerdo, los bits no significan nada.

Unidades de información

Unidad	Equivalencia	Ejemplo
1 bit	0 o 1	Una respuesta sí/no
1 byte	8 bits	Un carácter ASCII
1 KB	~1.000 bytes	Un párrafo de texto
1 MB	~1.000 KB	Una foto JPEG
1 GB	~1.000 MB	~250 canciones MP3
1 TB	~1.000 GB	~500 horas de video



Laboratorio analógico

"Pulseras Binarias"

Vamos a escribir nuestras iniciales en el código más antiguo de la informática.

Convención

- cuenta oscura / cuadro coloreado = 1 (encendido)
- cuenta clara / cuadro vacío = 0 (apagado)

Cada letra = **8 bits (1 byte)**

Se lee de **izquierda a derecha** (bit 7 → bit 0).

Fase 1 · Codificación (20 min)

Tarea: codifica tus **dos iniciales** (nombre + apellido) en binario.

1. Busca cada inicial en la **tabla ASCII** → número decimal
2. Convierte decimal → binario de **8 bits**
3. Colorea los cuadros en el papel (o ensarta cuentas)

Ejemplo: "H" = 72 = **01001000**, "P" = 80 = **01010000**

⚠ Siempre 8 bits. No olviden los ceros a la izquierda.

Fase 2 · Intercambio (15 min)

Tarea: intercambia tu hoja/pulsera con un compañero de **otro grupo**. Decodifica sus iniciales.

1. Lee los cuadros → escribe la secuencia de 0s y 1s
2. Separa en **dos bytes** de 8 bits
3. Convierte binario → decimal
4. Busca en la tabla ASCII → letra

Regla: NO puedes preguntar cuáles son sus iniciales.

Verificar al final: ¿coinciden?

Fase 3 · Mensaje secreto (15 min)

Tarea grupal: codificar una **palabra de 5 letras** y enviarla a otro grupo.

- Cada integrante codifica **una letra**
- Ordenan y entregan la secuencia completa
- El grupo receptor decodifica y anuncia la palabra

Solo mayúsculas, sin tildes ni ñ (ASCII básico).

Discusión

¿Cuántas cuentas necesitaron para una sola letra?

*8. Y un emoji necesita 32 (4 bytes). La información tiene un **costo** en espacio.*

¿Qué hubiera pasado si cada grupo usara una convención de colores diferente?

Caos. Sin un estándar compartido, la comunicación es imposible. Por eso existen ASCII y UTF-8.

¿Y si quisiéramos codificar la ñ?

No está en ASCII básico. Necesitamos UTF-8 → más bits por carácter → más cuentas.

Lo que aprendimos hoy

- Los computadores usan binario porque es **físicamente simple y robusto**
- Un **bit** es la unidad mínima de información (0 o 1)
- **Todo dato** — número, texto, imagen, sonido — se codifica como secuencias de bits
- La codificación no es intrínseca: es una **convención humana** (ASCII, RGB, WAV)
- Sin estándar compartido, los bits no significan nada

Próxima semana

Semana 3 · Teoría de la información: midiendo la sorpresa

Si cada letra cuesta 8 bits, un libro cuesta millones. ¿Se puede ser más eficiente? Eso es compresión — y tiene que ver con la sorpresa y una fórmula que ya conocen de ecología.

Dato: la fórmula de Shannon para medir información es la **misma** que el índice de diversidad H' .

¿Preguntas?

Semana 2 · Hablando en ceros y unos



Control 1

Conversiones binarias y codificación

15 minutos · Individual · Pueden usar la tabla ASCII